AIOps Module 4 Assignment 5: SwiftParcel ETA Service

Rohan Sai P. (DA24B049) 8 November 2026

0. Machine and reproduction

All results come from one run of `run_all.sh` on the machine below. The script writes every output file this report cites.

Machine	Ubuntu (aarch64) VM in UTM/QEMU on an Apple Silicon Mac; 4 vCPU, 7.2 GiB RAM
Software	Python 3.14.4, fastapi 0.162.1, uvicorn 0.54, scikit-learn 1.9.1, pandas 3.1.1, mlflow 3.18, ray 2.60
Load generator	`load_test.py` (given), run inside the same VM as the server
Ray	started with `num_cpus=2`, as the assignment requires

To reproduce from a clean checkout:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && bash run_all.sh
```

Outputs: `machine_info.txt`, `partA/*.json`, `partB/*.txt`, `partC/verify_output.txt`, `partD/smoke_output.txt`.

Reused lecture material. I adapted ideas from the Lecture 10 notebooks:

- Part A follows `rest_app.py` (REST_API_Serving_Exercise): bundle loaded once, pydantic `Field` ranges, timing middleware, a plain `def` endpoint.
- Part B follows `mlflow_register.py` and the Solution B shadow loop (MLflow_Serving_Exercise): pyfunc wrapper, float64 signature, alias calls.
- Part C follows `ray_serve_app.py` (Ray_Serve_Exercise): the deployment, router and `build` structure, and the `extra_work_ms` parameter.
- Part D follows the `smoke_test.py` check list from the Minikube_Serving slides: healthz, version, predictions, p95, bad input returns 422.
- `eta_data.py` and `load_test.py` are the given starter files.

1. Part A: REST service

`partA/eta_app.py` meets all seven requirements, and the 16 tests in `partA/test_eta_app.py` pass. One worker saturates at concurrency 2 with about 660 req/s; a second worker raised that to 1195 req/s.

Design. The model bundle loads once at import; `ETA_MODEL` selects v1 or v2. Two pydantic classes hold the field ranges, the 1 to 500 batch limit, and the cross-field rule as a `model_validator`, so an infeasible row returns the same 422 body as other errors. A middleware sets the request ID and times every request. `/metrics` prints Prometheus text: a counter by path and status, and a latency histogram with cumulative buckets, `+Inf`, `_sum` and `_count`. Only the event-loop middleware updates these counters, so no lock is needed.

Tests. The invalid-input test runs six cases: distance and hour out of range, rain not 0 or 1, weight 0, a wrong type and a missing field. The 500 test uses `monkeypatch` to replace the model with one that raises, and checks the body leaks no internals.

A3.1 Load test (v1, 1 worker, batch 1, 4 s per level)

Concurrency	Requests	RPS	p50 (ms)	p95 (ms)	p99 (ms)	Errors
1	2259	564.8	1.7	1.9	2.2	0
2	2647	661.8	3.0	3.5	4.8	0
4	2337	584.2	6.3	11.2	14.9	0
8	2215	553.8	13.5	24.8	31.7	0
16	2178	544.5	27.8	49.6	64.6	0

A3.2 Saturation point and capacity under the SLO

The worker saturates at concurrency 2. Throughput peaks there at 661.8 req/s and does not rise after it, while p50 latency doubles with each doubling of clients. Little's Law, $L = \lambda W$, shows the extra clients queue: at $c = 16$, $544.5 \times 0.0278 = 15.1$ requests in flight; at $c = 8$, $553.8 \times 0.0135 = 7.5$. Both match the client count; p50 slightly underestimates the mean latency. The service time is $1/661.8 \approx 1.5$ ms per request. Past saturation, throughput drops 18% (662 to 545 req/s), because load-generator threads compete with the server for the same vCPUs.

Capacity under p95 $\leq$ 80 ms: 662 req/s at peak, 545 req/s sustained at 16 clients (p95 49.6 ms). Every level meets the SLO, so the limit is the throughput of one process. With p95 at about 1.8× p50, Little's Law predicts p95 crosses 80 ms near 25 clients. This is a prediction; I did not measure it.

A3.3 Cost per row, batch 1 vs batch 100 (concurrency 1)

Batch	RPS	Rows/s	ms per row
1	561.0	561	1.78
100	423.2	42,320	0.024

A batch of 100 costs 0.6 ms more than one row, so each row is 75× cheaper. Most of the 1.8 ms is fixed per-request work: HTTP parsing, JSON decoding, pydantic validation, building a DataFrame and one scikit-learn call. Ridge on 100 rows is one matrix product that takes microseconds. Batching spreads the fixed cost over many rows, at the price of latency for the first row.

A3.4 Change: two uvicorn workers

Prediction: one worker is bound to one core by the GIL, and the VM has four vCPUs, so a second process should double capacity.

Concurrency	RPS, 1 worker	RPS, 2 workers	Speed-up	p95, 1 worker (ms)	p95, 2 workers (ms)
1	564.8	542.2	0.96×	1.9	2.2
2	661.8	1004.0	1.52×	3.5	2.2
4	584.2	1141.5	1.95×	11.2	4.8
8	553.8	1162.5	2.10×	24.8	11.1
16	544.5	1195.0	2.19×	49.6	24.5

Result: capacity roughly doubled. Peak to peak, 662 to 1195 req/s is 1.81×. The 2.19× at $c = 16$ overstates the gain, because one worker had fallen below its own peak by then. At $c = 1$ there is no gain: one sequential client cannot keep two workers busy. The shortfall from 2× comes from the load generator using CPU in the same VM. p95 at $c = 16$ halved, from 49.6 to 24.5 ms.

A3.5 Why p95/p99 and not the mean

The mean hides the tail: a 2 ms mean can coexist with one request in twenty taking ten times longer from queueing or GC (garbage collection) pauses. Users notice those slow requests, and one app action often makes several calls, so the tail of each call sets the speed of the whole action.

2. Part B: MLflow registry, gate and shadow

The gate promotes v2 under the default thresholds and blocks it at `min_rel_gain = 0.60`. In shadow, the two models differ by 9.48 min on average at equal latency.

B1 Registry

`python partB/eta_mlflow.py register` uses a SQLite tracking store. `EtaPyfunc` wraps a bundle and returns a DataFrame with `eta_minutes` (1 decimal) and `model_version`. Each model is logged with an inferred signature, a two-row input example and exact `pip_requirements` pins for mlflow, scikit-learn, pandas, numpy and joblib. Inputs are cast to float64 so the schema accepts JSON integers.

Registry version	Alias	Bundle model_version
1	prod	v1 (Ridge)
2	candidate	v2 (GradientBoosting)

B2 Promotion gate

`decide()` is a pure function: metric dicts in, decision out, no I/O, inputs unchanged. Slice regression is relative to prod's slice MAE: cand_slice/prod_slice − 1 ≤ max_slice_regression, so 0.05 means at most 5% worse. A value on a threshold passes. `promotion_gate()` loads both aliases, evaluates them on `make_data(2000, seed=99)` and times 200 single-row `predict` calls. The 10 tests in `partB/test_gate.py` place values just inside and just outside each threshold.

Metric on fresh data	prod (v1)	candidate (v2)
MAE overall (min)	11.955	6.105
MAE, rush hours (min)	19.680	8.580
MAE, rain = 1 (min)	11.623	6.285
Single-row p95 (ms), default run	1.232	1.174

Thresholds	MAE gain	Cand. p95	Rush	Rain	Promote
Defaults (0.03, 10 ms, 0.05)	48.9% ✓	1.174 ms ✓	−56.4% ✓	−45.9% ✓	true
`min_rel_gain` = 0.60	48.9% ×	1.215 ms ✓	−56.4% ✓	−45.9% ✓	false

The full JSON is in `partB/gate_output.txt`. Each run re-measures p95, so the two runs differ.

B3 Shadow comparison

Both aliases ran under `mlflow models serve` on ports 5101 and 5102. Each received the same 200 rows from `eta_fresh.csv`, one row per request.

Measure	Value
(i) Mean absolute difference	9.48 min
(ii) Requests differing by more than 10 min	35.0%
(iii) p95 latency, prod (5101) / candidate (5102)	3.6 ms / 3.5 ms

I would promote v2 through the canary, not a direct switch. It halves MAE, improves both slices and adds no latency. Most of the 35% large disagreements are v1 errors, since v1's MAE is 12 min overall and 20 min at rush hour. Still, a third of customers would see their ETA change, which calls for a sticky canary and monitoring.

Moving an alias does not change a running server. `mlflow models serve -m models:/eta-model@prod` resolves the alias to a version once, at startup, loads that version's artifacts into memory and never queries the registry again. A running service can take the new version without dropping requests in two ways:

1. **Rolling or blue-green restart:** start new server processes that resolve the moved alias, wait for `/ping` and the smoke test, shift the load balancer, then drain the old processes. A Kubernetes rolling update with readiness probes does this.
2. **In-process hot swap:** a background thread polls the alias version, loads the new model beside the old one and swaps the reference. A Ray Serve redeploy works this way: it starts new replicas before it stops the old ones.

3. Part C: Ray Serve sticky canary

All five checks in `partC/verify_ray.py` pass on a 2-CPU Ray: 20.3% of customers land on v2 at weight 0.2, and no customer changes arm except when the weight rises.

Design. `partC/eta_ray.py` defines three deployments:

Deployment	Replicas	CPU each	Role
Router (ingress)	1	0.1	buckets `customer_id`, forwards to v1 or v2, adds `arm` to the response
EtaModelV1	2, fixed	0.25	serves v1
EtaModelV2	1 to 3, autoscaling	0.25	serves v2

Each model replica loads its bundle once. The worst case reserves $0.1 + 2 \times 0.25 + 3 \times 0.25 = 1.35$ CPU, which fits in 2. The v2 `autoscaling_config` sets `target_ongoing_requests = 2`, `upscale_delay_s = 3` and `downscale_delay_s = 30`. The Router maps each customer to a fixed number in $[0, 1)$ and sends it to v2 when that number is below the weight w; `X-Beta: 1` forces v2:

$$\text{bucket}(id) = \frac{\text{int}(\text{sha256}(id)[0{:}8],\ 16)}{2^{32}}, \qquad \text{arm} = \begin{cases} \text{v2} & \text{bucket} < w \\ \text{v1} & \text{otherwise.} \end{cases}$$

A customer's bucket never changes; only w moves. Raising w can only move customers from v1 to v2.

Verification (`partC/verify_output.txt`)

Check	Result
CPU reserved at start	0.85 of 2
(a) Share on v2, 1000 customers, $w = 0.2$	20.3% (allowed 16 to 24%) ✓
(b) 200 customers, second call	200/200 same arm ✓
(c) After `serve.delete` and an identical redeploy	200/200 same arm ✓
(d) Weight $0.2 \to 0.5$	51.3% on v2; 0 of 203 former v2 customers moved back; 310 moved to v2 ✓
(e) `X-Beta: 1` from 50 v1 customers	50/50 served by v2 ✓

Each check is an `assert`. Check (e) also re-sends 5 requests without the header and confirms they return to v1.

C3 Short answers

1. Python salts `hash()` of a `str` with a random per-process seed (`PYTHONHASHSEED`). The same ID then hashes to a different bucket in each replica, after each restart and on each machine, and the customer switches arms. SHA-256 gives the same bucket everywhere; check (c) depends on this.
2. Product: a customer who refreshes the tracking page and sees 40 min, then 52 min, stops trusting the ETA. One model per customer keeps the number stable. Analysis: the customer is the unit of randomisation, so per-customer

outcomes such as complaints, late deliveries and re-orders trace to one arm. A per-request split exposes every customer to both models and breaks the independence a comparison needs.

3. A single-row prediction takes about 2 ms, so a target of 2 ongoing requests keeps each v2 replica busy while the queue, and p95, stay far below 80 ms. `max_replicas = 3` caps v2 at 0.75 CPU, so the app stays within 1.35 of 2 CPUs and leaves room for v1, the proxy and the controller. A 3 s upscale delay reacts to bursts; a 30 s downscale delay avoids adding and removing replicas on short dips.

4. Part D: release gate

`partD/smoke_eta.py` passed 7 of 7 checks against the Part A v1 server (`partD/smoke_output.txt`). It exits 0 when every check passes and 1 otherwise, so a deploy pipeline can stop on failure. Run: `python partD/smoke_eta.py --url http://localhost:8000/v1/models/eta:predict --expect-version v1 --max-p95-ms 80`.

#	Check	Passes if	Result
1	`GET /healthz`	status 200 and `"status": "ok"`	ok
2	Version and shape, 3 deliveries	expected `model_version`, 3 predictions in (0, 2000) min	v1, [21.1, 90.7, 245.4]
3	Sanity: distance 2, 10, 40, 120 km	ETA never decreases	18.5 to 322.8 min
4	Sanity: traffic index 0 to 0.9	ETA never decreases	52.1 to 90.6 min
5	Sanity: 1, 5, 10, 20 stops	ETA never decreases	47.6 to 129.8 min
6	`n_stops: "many"`	HTTP 422	422
7	p95 of 30 single-row requests	≤ 80 ms	2.6 ms

A crash inside a check counts as a failure, so every check is reported.

Abort criteria for a 10% sticky canary of v2

Each metric compares the v2 cohort with the v1 cohort over the same window.

#	Metric	Abort if	Window
1	5xx rate on v2 requests	$> 1\%$ absolute, or $> 2\times$ the v1 rate	5 min rolling, at least 200 v2 requests
2	p95 latency of v2	> 80 ms, or $> 1.5\times$ v1's p95	two consecutive 5 min windows
3	MAE on completed deliveries	v2 MAE $>$ v1 MAE by 5%, overall or in the rush-hour slice	rolling 1 h, at least 100 deliveries per arm
4	Implausible predictions (≤ 0 or > 600 min)	3 or more in v2, or rate $> 0.1\%$	10 min rolling

5. Not tested and limitations

- The load generator and the server share the VM's vCPUs, so measured throughput is a lower bound for a dedicated server.
- `mlflow models serve` ran only with `--env-manager local`. I did not test the pinned `virtualenv` path or `build-docker`.
- Ray ran on a single node. The v2 autoscaling config is set and its replica count is printed, but `verify_ray.py` does not exercise scaling under load.
- The Part C router checks field types and batch size, not the Part A value ranges or the infeasible rule.
- I ran `smoke_eta.py` only against the Part A v1 server. I did not run it against v2, the Part C router or the MLflow `/invocations` server, which uses a different payload.
- Tests and manual reading checked the `/metrics` output; no Prometheus server scraped it.